エージェント開発プロセス

agent-dev-template の全体像と役割

人間と AI で進める開発の全体像

スクラムを実践できるよう、**役割とイベント**を取り入れる



人間

方向・優先順位・採用を判断

対話して具体化する



PO AI

対話の結果を Issue にする

作成・更新

GitHub Issue

開発可で着手

計画

開発者 → 開発チーム

人間が承認

実装・検証

作って、確かめる

開発者どうしでレビュー

成果の確認

AI の PO が受け入れ確認

↓

人間が確認・採用判断

ふりかえり

開発チームが改善案

人間が採否を判断

結果を申し送り



開発者 AI

2人が1つずつ案を作る

AI の PO：結果を受け取り、次の仕様・Issue へ反映



開発チーム AI

この流れ全体を管理

人間と AI の役割分担



人間 ステークホルダー・PO

- 作る方向と優先順位を PO と相談して決める
- 計画・成果・改善案を確かめて判断する

⇄
対話して
具体化する



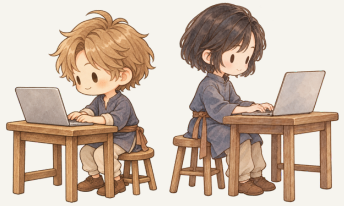
PO AI

- 人間と優先順位を検討し、仕様・Issue に整理する
- 成果を受け入れ条件と照らして確認する



開発チーム AI

- Issue ごとの進行を管理し、2つの実装案を比べる
- 採用後にマージ・後片付け・ふりかえりを行う



開発者 AI

- 同じ Issue を2人が別々に計画・実装・検証する
- 互いの PR をレビューする

AI は PO・開発チーム・開発者の3つの役割。独立したスクラムマスターは置いていない

リファインメント：やりたいことを仕様と Issue にする



人間

やりたいことを
PO と相談し、
方向と優先順位を決める

対話して
具体化
⇄



PO AI

相談の結果を
仕様（spec.md）に書き、
今回作る範囲を
Issue に切り出す

書く
→

Issue（PBI）

スクラムのプロダクト
バックログアイテム
（PBI）の代わり。
目的・範囲・
受け入れ条件を書く

PO が着手順を決めて「開発可」ラベルを付けると、開発チームが起動し、プランニングへ

スクラムでは開発者もリファインメントに加わるが、
このプロセスでは人間と AI の PO が中心で、開発者はプランニングから加わる

プランニング：何をどう作るかを合意する

スプリント期間は設けず、Issue ごとにスプリントの流れを踏襲する



開発者 AI

同じ Issue を読み、
自分の計画を立てる



開発者 AI

同じ Issue を読み、
自分の計画を立てる

計画
（2つ）
→



開発チーム AI

2つの計画と
確認したいことを、
人間に示す

計画
→



人間

作りたいものと照らして
確認・調整し、承認する

↩ 仕様の穴が見つかったら、
PO に戻して決め直す

人間が承認すると、開発者2人がその計画でそれぞれ実装・検証に進む（開発へ）

計画の承認を AI の PO に頼む手順はない。デイリースクラムは取り入れていない

開発：2人がそれぞれ作り、互いにレビューする

人間が承認した計画で、同じ Issue から2つの案を作る



開発者 AI

自分の計画で
実装・検証し、
PR（案1）を出す

互いの
PR を
レビュー
⇄



開発者 AI

自分の計画で
実装・検証し、
PR（案2）を出す

PR（2つ）
と
レビューの
結果
→



開発チーム AI

2つの案と
レビューの結果を
受け取り、比べる

比べた結果から、開発チームが推薦する PR を選び、レビューへ

新しい会議や承認は加えない。2人で作る進め方は任意で、1人で1つの案を作ることもある

レビュー：成果を確かめ、採用を決める

開発者どうしのレビューを終えた成果を、AI の PO → 人間の順に確かめる



開発チーム AI

2つの案と
レビューの結果を比べ、
推薦する PR を選ぶ

推薦
→



PO AI

受け入れ条件・仕様と
照らして受け入れ確認
（コードは見ない）

↩ 差し戻しなら、
開発チームが直して再確認

確認の
結果
→



人間

実物を確かめ、
採用か、修正か、
条件の見直しかを決める

↩ 条件を見直すときは、
PO と仕様・Issue を直す

採用が決まると、開発チームがマージと後片付けをして、ふりかえりへ（実行内容は人間が承認）

開発者どうしのコードレビューは開発の中の作業。このレビューは成果の確認で、別のもの

ふりかえり：進め方を改善し、次へつなぐ

採用した成果を取り込んだあと、Issue 1本の進め方をふりかえる



開発チーム AI

検証・レビューの
記録からふりかえり、
根拠のある改善案を示す

改善案
→



人間

改善案ごとに
採否を判断する

採用した
改善
→



開発チーム AI

採用した改善を
手順・文書へ反映し、
仕様についての発見を
PO へ申し送る

申し送りを受けた AI の PO が、人間と次の仕様・Issue を検討する（リファインメントへ）

反映した改善の効果は、次の Issue で確かめる。採用・反映・効果の確認は別のこと

ここから実例

実例：1つの Issue が進むまで

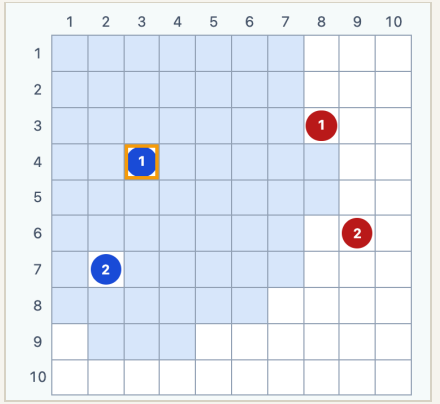
別の開発プロジェクト overstrata-board の Issue #37 を、ここまでの流れに沿って追う

この章の題材: overstrata-board #37

Issue（今回作る範囲と完成条件）の目的から（要約）

何のアプリか	マス目の盤面に駒を並べ、各駒が動ける範囲の目安を表示するアプリ（スマホ）。使うのはゲームの進行役
今の課題	用意した駒の移動力は、すべて5マス固定。駒ごとに変えられない
したいこと	準備中に駒ごとの移動力を設定し、その値で範囲の目安を出したい

画面と言葉



選んだ駒の範囲の目安（薄い青）

#37 を作る前の画面（全駒5マス）

- **移動力**: 1回に動けるマス数
- **準備中**: 駒を並べ、「確定」を押すまで
- **範囲の目安**: 選んだ駒の周りの薄い色。範囲の外へも動かせる

何を満たせば完成か（受け入れ条件の抜粋）

board #37 の受け入れ条件（10項目）から（要約）

設定の範囲	準備中に選んだ駒を、0〜9マス（初期値5）で設定する
操作	数値は表示だけ。「−」「＋」で1ずつ変える
反映	対象の駒と、範囲の目安へすぐ反映する
置き場所（当初）	設定欄は**対象の駒の近く**に出す。どの駒か分かり、対象だけ変わる

この条件への2つの解決案

- 開発者2人が、同じ課題と条件から別々に作った
- 分かれたのは設定欄の置き場所:**駒の近く案**と**操作パネル案**
- 置き場所の条件は、2案を比べた後に人間の判断で変わる（⑥で説明）

置き場所は、条件を変更する前の版

① 人間と PO が、作りたいものを仕様にする

このプロセスでは

担当	 PO  人間
渡すもの	spec.md：作るもの全体の仕様 （何を・なぜ・どこまで作るか）
人間が決めること	作りたいものを PO と議論する （3つの判断の前の段階）

実例: overstrata-board #37

- 前の Issue #32 から、移動力の設定を別の Issue（#37）に分けると、人間と PO が合意した
- PO が spec.md に記録: 0〜9マス・初期値5・設定欄は駒の近く

spec.md 2.2・6章、#37 レポート 2章

②③ 今回作る範囲を Issue にし、着手する

このプロセスでは

担当	 PO → GitHub Issue と「開発可」 → Orca の起動係 定期実行で起動 →  開発チーム
渡すもの	Issue（目的・受け入れ条件・スコープ外）と「開発可」
人間が決めること	この工程で追加の判断はなし 人間と相談して決めた優先順位を、PO が Issue の着手順に具体化する

Orca: AI の端末と worktree を扱う作業環境

実例: overstrata-board #37

- 範囲: 前提で示した条件を含む受け入れ条件10項目
- スコープ外: 戦闘途中の駒の増減、倒れた駒の表示（別の Issue）
- #32 の完了後に「開発可」。Orca の起動係が開発チームを起動し、19:45 に着手した

#37 レポート 2・3章

④ 開発者が計画を出し、人間が承認する

このプロセスでは

担当	 開発者  開発チーム
渡すもの	計画（受け入れ条件ごとの 作り方・確かめ方・確認したいこと）
人間が 決めること	 計画を承認する

実例: overstrata-board #37

- 2人の計画で、設定欄の置き場所が分かれた: **駒の近く案**と**操作パネル案**
- 人間は「それぞれの案のまま作り、画面で見比べる」と承認
- 操作パネル案は「駒の近く」を満たさないおそれ → 距離を測らせる

#37 レポート 3章

⑤⑥ 開発者が実装・検証し、互いにレビューする

このプロセスでは

担当	 開発者2人（それぞれのworktree）
渡すもの	PR（何をしたか・確認結果）と、相手のPRへのレビュー
人間が決めること	なし（この間は待つ）

実例: 触る前 → 味方2を選んだ直後

駒の近く案

PR #41

欄が一覧の中に入り、並びが組み替わる



操作パネル案

PR #40

一覧の上に欄が1つ出る。並びは2行のまま



⑥ 推薦・PO の受け入れ確認と、人間の採用判断

実例: #37 の往復（担当とすること）

開発チーム	**推薦**: PR #41（駒の近く案）
PO	**受け入れ確認**: #41 を受け入れ
人間	**採用判断**: 触り比べて #40 を選ぶ
PO・開発者	置き場所の条件を更新／並行して README を修正
開発者・開発チーム	更新後の条件で再レビュー、推薦を #40 へ
PO	**受け入れ**（#40）→ 人間の承認後に⑦へ

置き場所の条件の変更

- 変更前: 設定欄は、対象の駒の近くに表示する
- 変更後: 操作パネルに1つ表示し、駒名・陣営色の丸・番号で対象を示す
- 画面は作り直していない。#40 の追加の修正は README の1行

#37 レポート 6・7章

⑦⑧ 採用した成果を取り込み、改善を次へ渡す

このプロセスでは

担当		開発チーム: 承認の後に マージ・後片付け・ふりかえり
渡すもの		改善案（根拠・反映先）と、 PO への申し送り
人間が 決めること		• マージの実行内容を承認する • 改善の採否を決める

実例: overstrata-board #37

- 人間の承認後、開発チームが #40 をマージ。#41 はブランチを残して閉じた
- 改善 F: 実機確認の結果は会話で聞き、開発チームが PR に代筆する
- 改善 G: 条件と合わない作りが選ばれたら、PO に仕様の更新を頼む
- 人間が F・G を採用し、手順書に反映

#37 レポート 8〜10章